TECHNICAL DOCUMENTATION

System Overview

System Title: **0dd – Binary String Validator**

Description:
The 0dd program is a rule-based input processor that validates binary strings based on whether they contain an **odd number of 0s**. The system accepts user input either manually or through file upload and classifies each string as **Accepted** or **Not Accepted** depending on the validation rules.

A string is considered valid if:

- It contains only **0**, **1**, or whitespace characters
- It contains an odd number of 0s

The program provides a terminal user interface using the Python curses library and allows users to:

- Test strings manually
- Upload text files containing multiple inputs
- View program information
- Exit safely

Developers:
Charl Emmanuel E. Mendez
Kriztan V. Sumbillo
BSCS-3A (Batch 2027)

Course:
CSAL102 – Mini Project
Instructor: Ma'am Nikka Ysabel Farofaldane

Necessary Installations

The program uses the curses library to create a text-based terminal interface. Since this library is not built into Windows by default, an additional package is needed.

The program requires you to type this command into your device command terminal to install it:

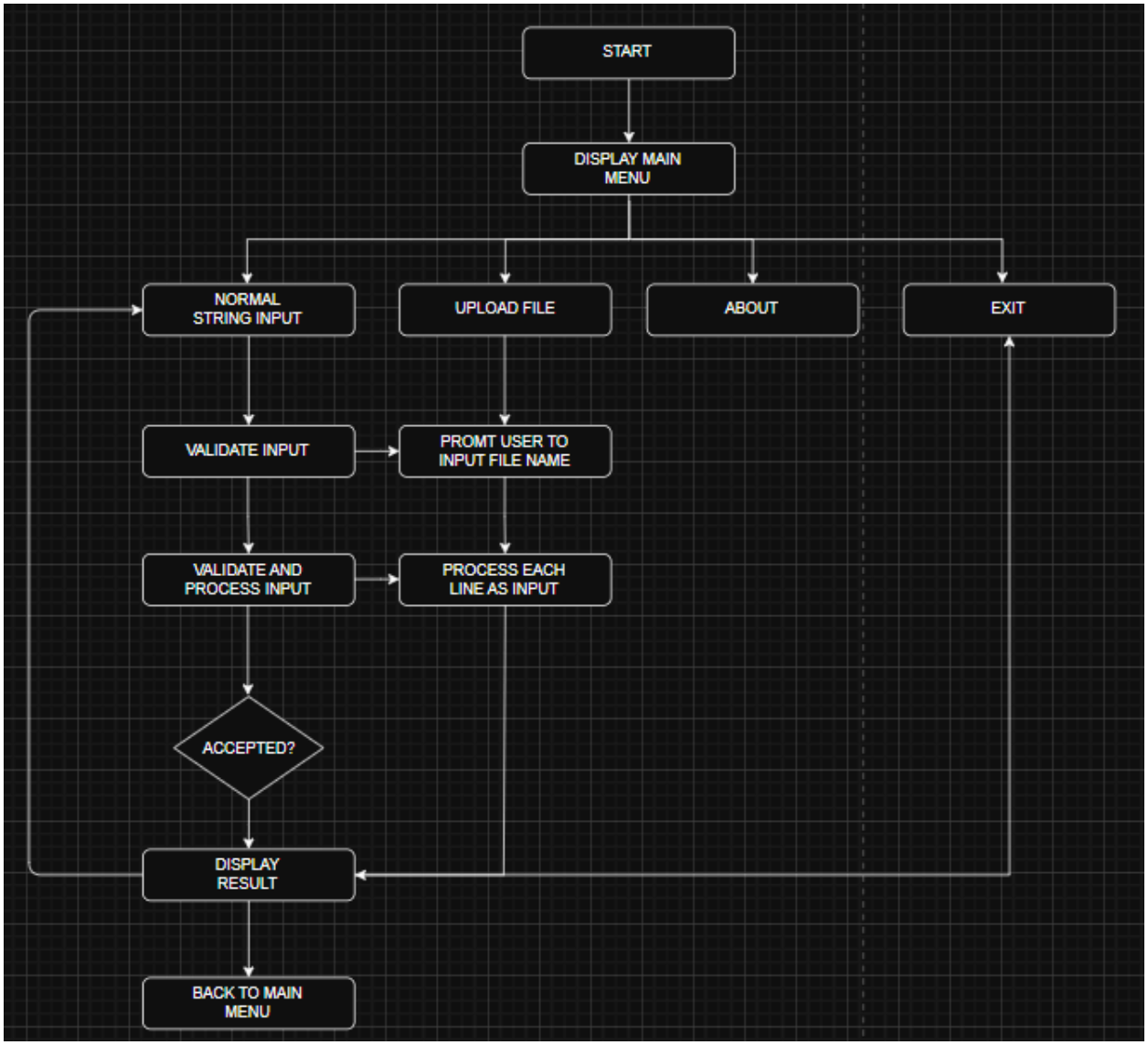
```
pip install windows-curses
```

Program Workflow

The general process of the system works as follows:

1. Program displays the **Main Menu**
2. User selects:
    - Normal Input
    - Upload File
    - About
    - Exit
3. If **Normal Input** is selected:
    - User enters a binary string
    - Program validates characters
    - Program counts zeros
    - Displays Accepted or Not Accepted
    - Program continuously accepts input unless user types Menu or Exit
4. If **Upload File** is selected:
    - User enters filename
    - Program reads each line
    - Each line is validated
    - Results are displayed
5. If **About** is selected:
    - Program displays project information
    - Returns to main menu after key press
6. If **Exit** is selected:
    - Program terminates safely

Flowchart



Program Architecture and Detailed Guide

The program contains the following major functions:

The program contains the following functions for it to work:

1. **main()** – the starting point of the program responsible for displaying the main menu and controlling navigation between the different features of the program.
2. **draw_frame()** – responsible for creating the border or frame of the terminal interface including the title and layout structure.

3. **process_string()** – checks the entered string and determines whether it is accepted or not based on the condition that the number of **0s** must be odd and that the string only contains **0s and 1s**.
4. **get_color()** – responsible for assigning the appropriate color to the result (green for accepted, red for not accepted, and white for errors).
5. **normal()** – allows the user to manually input strings and displays whether each input is accepted or not.
6. **upload()** – allows the user to input a filename, reads the file contents, and processes each string inside the file.
7. **about()** – displays the information about the program taken from the **about.txt** file.
8. **exit()** – responsible for properly terminating the program and displaying a thank-you message before closing.
9. **curses.wrapper()** – initializes the curses environment and safely runs the **main()** function while handling terminal setup and cleanup.

For a more detailed explanation, let's take a deeper look into the architecture and code of each function.

1. **main()**

The function responsible for controlling the main flow of the program. It is the starting point of the program and manages the main menu interface. It displays the list of options (Normal Input, Upload File, About, and Exit) and allows the user to navigate using the keyboard arrow keys. It stores the currently selected menu option in a variable named **curr** and updates it depending on the user's key presses.

This function also initializes the **curses color settings** which are used to display colored results in the interface. When the user presses the **Enter key**, it determines which menu option was selected and calls the appropriate function (**normal(), upload(), about(), or exit()**). After a selected function finishes, control returns back to the main menu, allowing the user to continue using the program until they choose the Exit option, which terminates the program.

```python
def main(stdscr):
    curses.curs_set(0)
    curr = 0

    while True:
        stdscr.clear()
        h, w = len(menu), 30
        for i, item in enumerate(menu):
            if curr == i:
                stdscr.addstr(i+3, 2, f"► {item} ", curses.COLOR_GREEN)
            else:
                stdscr.addstr(i+3, 2, item)

        draw_frame(stdscr, "0dd - Main Menu", h, w)
        stdscr.refresh()
        key = stdscr.getch()

        if key == curses.KEY_UP:
            curr = (curr-1) % h
        elif key == curses.KEY_DOWN:
            curr = (curr+1) % h
        elif key == curses.KEY_ENTER or key in (10, 13):
            selected = menu[curr]
            stdscr.clear()
            stdscr.refresh()

            if selected == "Normal Input":
                normal(stdscr, selected)
            elif selected == "Upload File":
                upload(stdscr)
            elif selected == "About":
                about(stdscr, selected+" 0dd")
            elif selected == "Exit":
                exit(stdscr)

curses.wrapper(main)
```

2. **draw_frame()**

The function responsible for designing the layout of the program interface. It creates the border of the terminal window including the top border, side borders, and bottom border. It also displays the title of the current screen and adjusts the frame size depending on the

height (**h**) and width (**w**) parameters passed to it. This function is repeatedly called by other functions to maintain a consistent interface design throughout the program.

```
def draw_frame(stdscr, title, h, w):
    stdscr.addstr(0, 0, "╔" + "="*(w-2) + "╗")
    stdscr.addstr(1, 0, title.center(w))
    stdscr.addstr(1, 0, "║")
    stdscr.addstr(1, w-1, "║")
    stdscr.addstr(2, 0, "╟" + "="*(w-2) + "╢")

    for y in range(3,h+3):
        stdscr.addstr(y, 0, "║")
        stdscr.addstr(y, w-1, "║")

    stdscr.addstr(h+3, 0, "╚" + "="*(w-2) + "╝")
```

3. **process_string()**

The function responsible for validating the input string. It removes unnecessary spaces and checks if the string is empty. It also verifies whether the string only contains binary digits (0 and 1). If valid, it checks whether the number of **0s** is odd to determine if the string is accepted or not. If invalid characters are detected, it returns an error message indicating that only binary inputs are allowed.

```
def process_string(s):
    s = s.strip().replace(" ", "")
    if not s:
        return "Empty string detected"
    if all(c in ('0', '1') for c in s):
        return "Accepted" if s.count('0') % 2 else "Not Accepted"
    else:
        return "Please input a string containing only 0s and 1s"
```

4. **get_color()**

The function responsible for assigning the correct display color to the result of the processed string. It returns a green color if the string is accepted, red if not accepted, and white if the result is an error message. This helps visually distinguish the results in the terminal interface.

```
def get_color(result):
    if result == "Accepted":
        return curses.color_pair(1) | curses.A_BOLD
    elif result == "Not Accepted":
        return curses.color_pair(2) | curses.A_BOLD
    else:
        return curses.color_pair(3)
```

5. **normal()**

This function handles the manual input of binary strings from the user. It repeatedly asks the user to enter a string and saves the inputs in a list so that previous results remain visible on the screen. Each input is checked using the **process_string()** function to determine if it is accepted or not, and the result is displayed with colors using the **get_color()** function.

The function also allows the user to type special commands such as **menu** to go back to the main menu, **exit** to close the program, and **clear** to remove all previous inputs. It also adjusts the size of the interface depending on the number of inputs so the display will remain organized. If the terminal window becomes too small, the function shows an error message to inform the user.

```python
def normal(stdscr, title):
    try:
        inputs = []

        while True:
            try:
                w = max(54, len(max(inputs, key=len))+20)
            except:
                w = 54

            h = (len(inputs))*2
            i = 0
            stdscr.clear()

            with open("normal_input_ins.txt", "r") as file:
                i = 3
                for line in file:
                    if line == "\n":
                        i += 1
                        continue

                    splitted_text = textwrap.wrap("• " + line, width=(w-4))
                    for t in splitted_text:
                        stdscr.addstr(i, 2, t)
                        i += 1

            draw_frame(stdscr, title, h+(i-2), w)
```

```
        for inp in inputs:
            result = process_string(inp)
            stdscr.addstr(i, 2, "Enter a string: ")
            stdscr.addstr(inp, get_color(result))
            stdscr.addstr(i+1, 2, "  "+result, get_color(result))
            i += 2

        curses.echo()
        curses.curs_set(1)
        stdscr.addstr(i, 2, "Enter a string: ")
        s = stdscr.getstr(i, len("Enter a string: ") + 2).decode("utf-8")
        curses.curs_set(0)
        curses.noecho()
        stdscr.refresh()

        key = s.lower().strip().replace(" ", "")

        if key == "menu":
            return
        elif key == "exit":
            exit(stdscr)
        elif key == "clear":
            inputs.clear()
        else:
            inputs.append(s)

except curses.error:
    stdscr.clear()
    draw_frame(stdscr, "Error", 1, 54)
    stdscr.addstr(3,2, "Error: Terminal window is too small")
    stdscr.getch()
    upload(stdscr, title)
```

6. **upload()**

The function responsible for processing binary strings from external text files. It repeatedly asks the user to input a filename and stores the filenames entered in a list called **files**. The function attempts to open each file and reads its contents line by line. Each valid line is cleaned by removing spaces and then passed to the **process_string()** function to determine whether the string is accepted or not. The results are then displayed

together with the corresponding input string using color formatting through the **get_color()** function.

The function also keeps track of the longest output string to dynamically adjust the interface width for proper display formatting. It includes error handling to manage cases such as missing files, empty filenames, and invalid inputs. Similar to the normal input function, it also accepts special commands such as **menu** to return to the main menu, **exit** to terminate the program, and **clear** to remove previously entered filenames and results. It also displays instructions from a text file (**upload_input_ins.txt**) to guide the user on how to use the feature.

```
def upload(stdscr, title="Upload File"):
```

7. **about()**

The function responsible for displaying the program description. It reads the contents of the **about.txt** file and displays the information inside the terminal interface. After displaying the information, it waits for the user to press any key before returning to the main menu.

```
def about(stdscr, title):
    while True:
        h, w = 12, 54
        with open("about.txt", "r") as file:
            y = 3
            for line in file:
                if line == "\n":
                    y += 1
                    continue

                splitted_text = textwrap.wrap(line, width=(w-4))
                for t in splitted_text:
                    stdscr.addstr(y, 2, t)
                    y += 1

        stdscr.addstr(y+1, 0, "Press any key to return to Main Menu".center(w-4), curses.color_pair(1) | curses.A_BOLD)
        draw_frame(stdscr, title, y, w)
        stdscr.refresh()

        stdscr.getch()
        return
```

8. **exit()**

The function responsible for properly terminating the program. It displays a closing message thanking the user for using the program and waits for a key press before calling **sys.exit()** to stop the execution of the program.

```
def exit(stdscr, title="Exit"):
    stdscr.clear()
    draw_frame(stdscr, title, 1, 54)
    stdscr.addstr(3, 2, "Thank you for using our program!")
    stdscr.getch()
    sys.exit()
```

9. **curses.wrapper()**

The function responsible for initializing and safely managing the curses environment. It automatically sets up the terminal for curses operation and calls the **main()** function. It also ensures that the terminal settings are restored properly when the program exits, preventing display issues after program termination.

```
curses.wrapper(main)
```

Example Input/Output

1.) NORMAL INPUT

a.) Main Menu

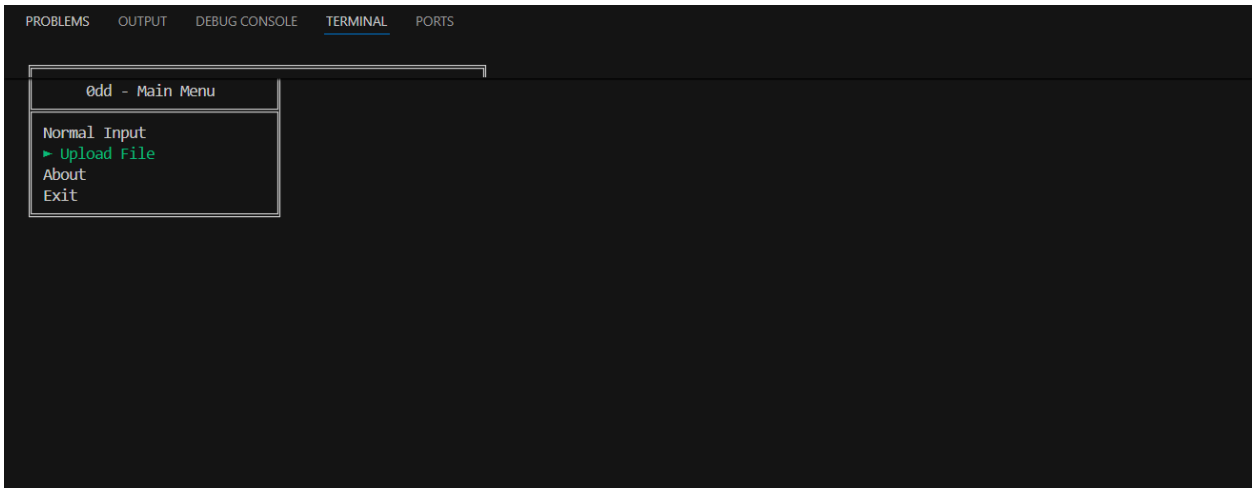
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

        0dd - Main Menu

  ► Normal Input
  Upload File
  About
  Exit
```

b.) Input & Results

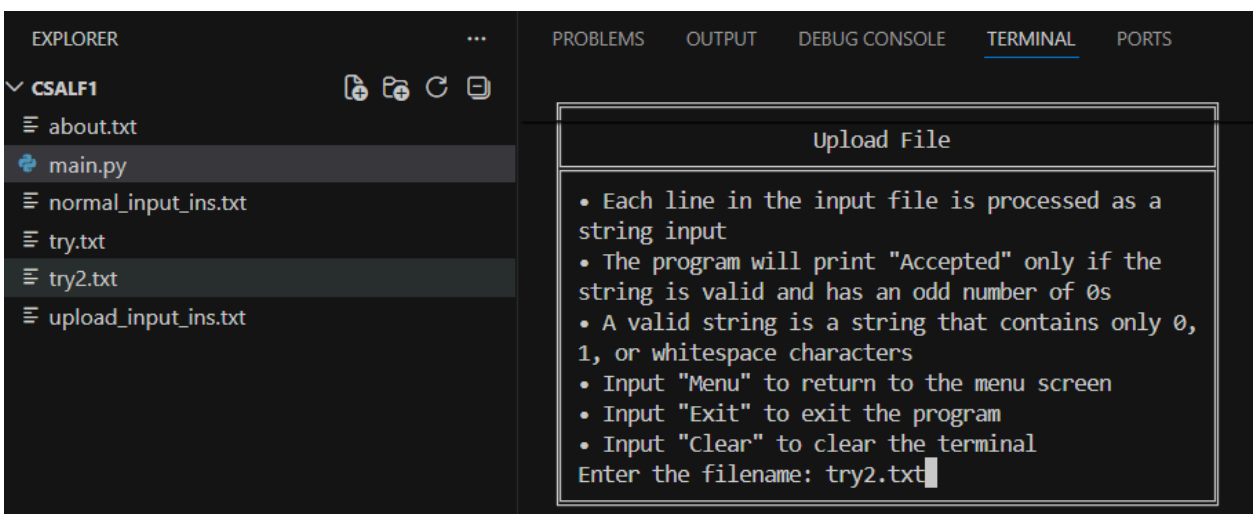
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

                    Normal Input

  • This program will say "Accepted" only if the
  inputted string is valid and has an odd number of
  0s
  • A valid string is a string that contains only 0,
  1, or whitespace characters
  • Type "Menu" to return to the main menu
  • Type "Exit" to exit the program
  Enter a string: 1000
    Accepted
  Enter a string: 101
    Accepted
  Enter a string: 10000
    Not Accepted
  Enter a string: abc1
    Please input a string containing only 0s and 1s
  Enter a string: 120
    Please input a string containing only 0s and 1s
  Enter a string:
```

2.) UPLOAD FILE

a.) Main Menu



b.) Inputting the existing file



Welcome | main.py | try2.txt | try.txt

try2.txt

```
100
11111
1222
100000
0000
00000000
zxc
1234.,
```

c.) Result

```
Upload File

• Each line in the input file is processed as a string
input
• The program will print "Accepted" only if the string is
valid and has an odd number of 0s
• A valid string is a string that contains only 0, 1, or
whitespace characters
• Input "Menu" to return to the menu screen
• Input "Exit" to exit the program
• Input "Clear" to clear the terminal
Enter the filename: try2.txt
  100: Not Accepted
  11111: Not Accepted
  1222: Please input a string containing only 0s and 1s
  100000: Accepted
  0000: Not Accepted
  00000000: Not Accepted
  zxc: Please input a string containing only 0s and 1s
  1234.,: Please input a string containing only 0s and 1s
Enter the filename:
  File not found
Enter the filename:
```

If file not existing:

```
Enter the filename: try3.txt
  File not found
Enter the filename:
```

3.) ABOUT

a.) Main Menu

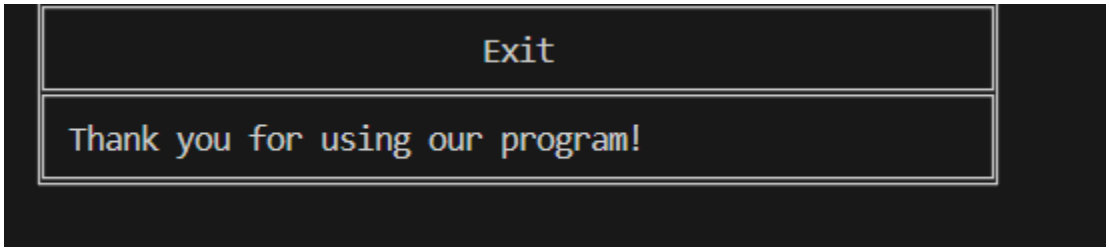
```
      0dd - Main Menu

Normal Input
Upload File
► About
Exit
```

b.) Result

```
About 0dd

This project was developed by Charl Emmanuel E.
Mendez and Kriztan V. Sumbillo of BSCS-3A (Batch
2027) as the Final Project for CSAL101 under Ma'am
Nikka Ysabel Farofaldane.

This program is a rule-based input processor that
allows users to test examples, upload files, and
experiment with custom rules for pattern matching
and validation.

     Press any key to return to Main Menu
```

4.) EXIT

a.) Main Menu

```
0dd - Main Menu

Normal Input
Upload File
About
► Exit
```

b.) Result